

[C언어 5일차 과제_전서연]

// 로빛 C언어 강의 자료와 학교 C프로그래밍 강의 자료를 복습하였습니다.

1. 포인터 기초

1) 포인터 정의

- 어떤 값이 저장되어 있는 메모리의 주소를 저장하는 변수 또는 상수
- (자료형) * (포인터의 이름);으로 정의한다.
- 포인터가 가르키는 메모리 영역의 크기는 sizeof(자료형)이 된다.
- 여러 개의 포인터 변수를 선언할 경우, int *a, *b와 같이 각각 *을 붙여야한다.
- 다른 자료형의 포인터로 형을 변환할 수 있다.
- >int* p1= &a; 가 있었다면 int s1=sizeof(*(char*)p1)이 가능하다.

2. 포인터 관련 연산자

1) 포인터와의 연산

(1) 포인터 변수에 offset 값을 더하거나 뺄 수 있다.

- 포인터가 가리키는 자료형의 크기만큼 주소가 이동할 수 있다. (offset)*(자료형 크기)만큼 이동 가능하다.
- 배열에 사용할 경우, 범위를 벗어나게 연산하면 안된다.

(2) 타입이 같은 포인터의 뺄셈은 가능

- 연산 결과는 포인터가 가리키는 대상 주소의 차를 자료형 크기로 나눈 값이다.

(3) 불가능한 연산

- 포인터의 덧셈, 포인터를 피연산자로 취하는 *, /, %의 연산은 불가하다.

(4) 주소 대입 연산

- 두 포인터 p1, p2가 존재할 경우, p1=p2가 가능하다.
- > p1이 가리키던 주소값을 p2이 가리키던 주소값으로 바꾼다.

2) 참조(간접) 연산자 (*)

- 참조 연산자를 사용하여 포인터 변수가 가리키는 메모리 주소의 값을 읽거나 수정 가능하다.

3) 주소 연산자 (&)

- 변수의 주소 값을 얻을 수 있다.
- &a일 경우, 대상 주소는 a의 주소이고, 자료형은 a의 자료형에 대한 포인터이다.

4) 증감 연산자 (p가 포인터 변수일 경우, p++, ++p, p--, --p)

- p++: 연산 결과는 p, 이후 p의 대상 주소가 자료형 크기만큼 증가.
- ++p: 연산 결과는 대상 주소가 자료형 크기만큼 증가한 이후의 p

- p--: 연산 결과는 p, 이후 p의 대상 주소가 자료형 크기만큼 감소.
- --p: 연산 결과는 대상 주소가 자료형 크기만큼 감소한 이후의 p

5) 간접 멤버 연산자 (->)

- p->m (m은 식별자.): p가 가리키는 구조체의 멤버 m이 된다.

6) 첨자 연산자 ([])

- 배열 또는 배열의 포인터에 사용 가능하다.
- p가 포인터인 경우, p[i]의 연산 결과는 *(p+i)가 된다.

3. call by value와 call by reference

1) call by value (값을 복사)

- 함수는 매개변수를 전달할 때 값을 복사하여 전달한다.
- => 함수 내에서 변수의 값을 바꾸더라도 main에 반영이 안 된다.

2) call by reference (바꾸고 싶은 변수의 주소값 전달하여 원본 전달)

- 포인터를 사용하여 변수의 주소를 전달한다. 위 방법을 사용하면 함수 내에서 매개 변수의 원본 값을 바꿀 수 있다.

4. 배열과 포인터

1) 배열과 포인터의 관계

- 1차원 배열일 경우, *(p(배열 list의 주소)+(정수 i))=list [i]와 같은 방식으로 처리가 가능하다.
- 2차원 배열일 경우, (*(p+i)+j)=list[i][j]와 같은 방식으로 처리가 가능하다.
- 2차원 배열의 경우 추가적으로 아래와 같은 코드 작성이 가능하다.

int a[2][4] = {1,2,3,4,5,6,7,8}

int (*pA)[4]; (행의 개수를 알면 모아서 처리가 가능하기 때문이다.)

pA= a;

이후, pA[i][j]으로 사용 가능하다.

2) 배열을 가리키는 포인터의 정의 형식

- 먼저 배열을 정의한 후, 배열 이름과 기호를 괄호로 감싸면 된다.

ex) int parr[2]-> int (*parr)[2] (연산자의 우선순위 때문이다.)

- 배열의 이름은 배열의 주소를 가져오는 포인터가 된다. 이름에 연결된 주소값은 변경될 수 없다.

4) 포인터를 배열로 관리하는 법

```
int* a[2];
```

```
a[0]=&x;
```

```
a[1]=&y
```

 과 같은 방식으로 포인터를 배열로 관리할 수 있다.

5. 문자열과 포인터

-문자열도 배열로 취급되지만, 이는 포인터에 의해 변경될 수 없다.

`char s1[]="hello, world";` -> 한 칸씩 수정 가능, 처음 초기화된 배열의 크기 이상의 작업을 할 수 없다는 단점이 존재함.

`char* s1="hello, world";` -> 한 칸씩 수정 불가능, 새로운 배열의 포인터를 가리킬 경우 그 크기 이상의 작업을 할 수 있다.

6. 구조체 포인터

```
typedef struct{
    int x;
    int y;
}point;
int main(){
    point pt = {10, 20};
    point *pp=&pt;
}
```

왼쪽과 같은 코드에서, 원래대로 구조체의 멤버에 접근하려면,

`(*pp).x`와 같이 .으로 구조체에 접근하여야한다.

다만 2-5에서 언급한 간접 멤버 연산자를 통해 `(*pp).x`를 `pp->x`로 축약하여 사용할 수 있다.

만약 함수의 매개변수로 포인터 없이 구조체를 그대로 넘기면 함수 내에서는 복사본을 수정하는 것이라 원본까지는 바뀌지 않는다. 이러한 문제를 해결하기 위해 구조체 포인터를 사용한다.

7. 함수 포인터

1) 함수 포인터

- add라는 함수가 정의된 경우,

`int (*fp)(int, int);`// 괄호 없이 `int *fp(int, int)`할 경우, int 포인터를 리턴하는 함수라는 뜻이 되므로 주의하여야 한다.

`fp=add;` //add의 주소를 가리킴

위와 같이 사용할 수 있다. 이렇게 작성된 코드를 통해 함수를 배열이나 구조체에 담아 사용할 수 있다.

2) 배열과 함수

-함수의 인자로 배열을 넘기는 경우, call by reference를 이용한다. (배열의 주소값을 함수의 인자로 넘긴다는 뜻이다.)

-매개변수로 배열의 길이를 같이 전달하여 범위를 초과하지 않도록 조치해야한다. (매개변수로 들어간 배열의 크기는 무시되기 때문이다.)

-> 함수가 배열의 최대 인덱스를 알 수 있도록 코드를 작성하는 것이 실행 시 오류를 방지할 수 있다.

8. 이중 포인터

-함수 외부에서 정의된 포인터값인 주소값을 함수의 인수로 받아서 사용하고자 할 때 이용된다.

<참조, 주소 연산자, 이중 포인터의 이해하기 쉬운 정리>

☆ 포인터를 정의할 때의 *와 사용할 때의 *는 의미가 다르다

-int *p=&i;를 통해 p는 i의 주소를 가리킨다! (이건 포인터 타입이라는 문법)

-이후의 *p는 p가 가리키는 메모리 속 값인 i가 된다. (p가 가리키는 곳에 접근하는 연산자)

-int **pp=&p인 경우, pp는 p의 주소값

-*pp는 p를 읽어낸 값-> i의 주소값

-**pp는 i의 값이 된다.

9) 사용 이유

-함수 내부에서 외부의 값을 가져오거나 수정 가능하다.

(=> 원본 값을 함수 내에서 직접 수정 가능)

-함수에서 두 개 이상의 값을 반환해야 할 때 사용 가능하다.

-큰 데이터 구조를 간단히 참조 가능하다.

-동적 메모리 할당의 결과를 포인터로 받기 때문이다.

- 지난 과제인 연결 메모리 등을 구현할 때 필요하다. (다음 노드를 가리켜야 하기 때문)

- 배열, 문자열 처리를 효율적으로 다룰 수 있다.

10) 유의할 점

-포인터는 자신의 자료형에 맞는 주소값을 참조할 수 있다.

-초기화되지 않은 포인터에 접근할 수 없다.

-배열에 사용할 경우, 범위를 벗어나게 연산하면 안된다.

11) 이전 과제에서 사용된 포인터

07/21 6번 과제 일부

```
int main() {
    int *numbers = NULL; // NULL 저장 (후에 동적 할당 할 때 사용할 예정)
    int count = 0; // 몇 개의 숫자를 배열에 추가했는지 확인하는 변수
    for (int i = 0; i < 12; i++) {
        input_number(&numbers, &count); // 각각의 주소를 보냄 (함수 내에서 두 값을
        //직접 바꾸기 위해서.)
        print_shape(numbers, count); // 주소와 값을 보냄
    }
    free(numbers); // 메모리 해제
    return 0;
}
//메모리 할당 및 숫자 입력 함수
void input_number(int** numbers, int* count) {
    if (*numbers == NULL) {
```

```

        *numbers = (int*)malloc(sizeof(int));
        if (*numbers == NULL)
        {
            printf("메모리 할당 실패\n");
            exit(1);
        } // 메모리 할당 실패 시의 예외 처리
    }
    else {
        *numbers = (int*)realloc(*numbers, sizeof(int)*(*count+1)); //입력을 받을 때마
        다 메모리를 재할당함.
        if (*numbers == NULL)
        {
            printf("메모리 할당 실패\n");
            exit(1);
        }
    }

    int result_1;
    printf("숫자 배열에 사용할 음이 아닌 정수를 입력하십시오 :");
    result_1 = scanf("%d", *numbers+*count); // 주소에 값 입력
    if (result_1 != 1 || *(*numbers + *count) < 0) { // 입력이 숫자가 아니거나, 음수인 경우
        예외 처리하는 코드 *(*numbers + *count)가 입력 받은 수
        printf("음이 아닌 정수만 입력할 수 있습니다.");
        exit(1);
    }
    else {
        (*count)++; // count의 값을 증가
    }
}

//숫자 배열 출력
void print_shape(int* numbers, int count) {
    for (int row = 0; row < 3; row++) {
        for (int col = 0; col < 4; col++) {
            int x = (row*4 + col) % count; //전체 칸을 처음부터 끝까지 순서대로
            센 번호를 count로 나눈 나머지가 배열의 인덱스가 됨 (반복 구조이기 때문),
            printf("%d", numbers[x]);
        }
        printf("\n");
    }
}

```

위 코드를 보면, int ** numbers를 쓴 것을 볼 수 있다. 이를 통해 numbers라는 포인터 변수의 자체 주소 값을 받을 수 있다. 만약 단일 포인터를 썼다면, main의 numbers는 여전히 옛날 주소를 가리키고 할당된 새 메모리에 접근하지 못한다.

count에 대해서는 단일 포인터를 썼다. 왜냐하면 포인터가 가리키는 장소의 변화가 필요한 것이 아니라 count가 가진 메모리 안 값을 변경하는 것이기 때문이다.

scanf할 때 *numbers+*count를 사용한 이유는 scanf에서 주소를 필요로 하기 때문에 &을 쓰지 않고 위와 같이 표시한 것이다.

print_shape에서 단일 포인터를 쓴 것은 numbers가 가리키는 곳을 바꾸지도 않으며, 메모리를 재할당하는 것이 아니므로 위와 같이 사용하였다.

-본인은 포인터를 함수 내 매개변수로 사용하는 것을 어려워하였다. 하지만 21일 과제를 통해 이중포인터로 할당받았을 경우, 이중포인터까지 사용할 수 있다는 뜻이지 *number가 ***number와 같은 의미를 지니지 않는다는 것을 이해하였다.

7/22 과제에서는

```

void arr_ij(int* sizeRow, int* sizeCol, int** pArr);
void print(int* row, int* col, int** pArr); 라고 정의해둔 뒤
arr_ij(&sizeRow, &sizeCol, arr);

```

`print(&row, &col, arr);` 라고 main에 작성하는 것이 이해가 되지 않았다.

우선, 위와 같이 쓴 것은 함수가 요구하는 타입과 사용할 변수의 타입이 같아야 넘길 수 있기 때문이었다. 이 점은 이해하였으나, 다음으로 `int **arr`이라고 선언한 상태에서는 이미 이중 포인터로 설정하였으니, `arr`만 넘겨도 이중 포인터 역할을 한다고 생각하여 오개념을 학습하였다. 추후 이에 대해 알아보니, `int**arr`이라 해도, `arr`만 넘기면 `int**`의 값만 복사되어 넘어가므로 `arr` 자체를 바꾸고 싶다면 `&arr`을 넘겨야 함을 알게 되었다.